Consumer Choice and the Entry of DIGI: An Agent-Based Model of the Portuguese Market of Bundled Services with Learning Incumbents

André Pires, José Pinho, Manuel Sampaio, and Pedro Correia

Faculty of Economics, University of Porto, Porto, Portugal
up202401101, up202006314, up202503379, up202102440

Abstract. This report analyses the Consumer Choice model, an agent-based model in which consumers choose between products under the influence of habits, information from friends and social norms, and adapts it to the entry of DIGI into the Portuguese market of bundled services. The baseline was implemented in Python with Mesa 3 and replicated 1000 times. Clustering reaches 0.94 and the final market share varies widely between runs (standard deviation 0.25). The new model has four operators, prices, loyalty contracts with legal exit penalties, a quality measure built from complaints and a changing population. It is calibrated to the market shares published by ANACOM (*Autoridade Nacional de Comunicações*, the Portuguese communications regulator) and reproduces the slow growth of DIGI with a root mean square error (RMSE) of 0.22 percentage points. The three incumbents then learn their prices with an ε-greedy multi-armed bandit. Each incumbent would gain by cutting its price alone, but joint profit falls by 17.5% if all cut by 10%, a prisoner's dilemma. The learning incumbents return to the list price (the price of their main offer before the entry) in most decisions and lose only 2.4% of joint profit. A sensitivity analysis that changes six assumptions one at a time, in nine variants, confirms both conclusions.

Keywords: Agent-based modelling · Consumer behaviour · Social influence · Telecommunications · Reinforcement learning · Mesa.

1 Introduction

This report covers the six tasks of Project 1 of the Agentic AI (Artificial Intelligence) course. The tasks are to analyse a model and define a baseline, to run it many times to obtain reliable statistics, to fit it to a real situation, to support the new model with scientific articles, to extend it with agent learning, and to run, analyse and discuss the new model.

The chosen model is Consumer Choice [1], from the NetLogo Models Library [2]. It derives from a basic model by M. A. Janssen and from earlier work on habits and social processes in market dynamics [3]. It simulates a market with two competing products whose quality can change. Consumers are linked in a social network and

choose a product according to three mechanisms: habit, informative social influence (learning about quality from friends) and normative social influence (using the same product as the majority of friends) [4]. The model shows how these mechanisms produce clusters of consumption, tipping points and social lock-in, a situation in which a product of lower quality keeps its market because it is the norm [5].

The real case is the entry of DIGI into the Portuguese market of bundled services in November 2024. DIGI sells fibre and television for about half the price of MEO, NOS and Vodafone, with a commitment of 3 months instead of 24. Even so, its share, together with NOWO, which it acquired, grew only from 2.6% to 3.2% between the third quarter of 2024 and the first quarter of 2026 [6]. The new model asks why a much cheaper operator grows so slowly, and how the incumbents should respond on price. Habit, social influence and loyalty contracts are the mechanisms that can explain the first question, and the Consumer Choice model already contains the first two.

Both models were implemented in Python with the Mesa 3 framework [7]. The baseline follows the NetLogo code rule by rule. Section 2 describes it along the lines of the ODD (Overview, Design concepts and Details) protocol [8] and Section 3 presents the baseline experiments. Section 4 describes the new model, its data, its calibration and the learning incumbents. Section 5 presents the results of the new model and its sensitivity analysis, and Section 6 concludes. The code, data and figures are available in the project repository.

2 Model Description

2.1 Purpose and Entities

The purpose of the model is to explain how habits and social influence shape the market shares of two products, and why consumers may fail to adopt a product of higher quality. The model contains one type of agent, the consumer. The social network and the products are not agents (Table 1).

Table 1. Entities of the model.

Entity	Represents	Role in the model
Consumer (100)	A person who buys one of two products	The only agent type. It chooses a product, learns about quality and influences its friends
Link (≈250)	A friendship	Channel through which information and norms are transmitted. The network is fixed during a run
Product: orange (0) and blue (1)	Two competing brands	Defined by global parameters (`quality_orange`, `quality_blue`) that can be changed during a run

Time is discrete. In each tick all consumers first execute the consumption decision and then update their utilities. Within each phase the consumers are activated in random order (Listing 1). The outputs are recorded at the end of each tick.

```
126  def step(self):
127      self.agents.shuffle_do("consume")
128      self.agents.shuffle_do("update_utilities")
129      self.datacollector.collect(self)
```

Listing 1. Schedule of one tick (`consumer_choice/model.py`, lines 126 to 129).

2.2 State Variables of the Consumer

Each consumer holds five state variables (Table 2, Listing 2), and a position used only to build the network (Sect. 2.3). The variable `satisfaction` links product quality to habit: it is the probability that the consumer keeps its current product without considering the alternative. The variable `personal_experience` is the memory of the consumer. It is updated only for the product in use and may therefore become outdated when the quality of the other product changes.

Table 2. State variables of the consumer.

Variable	Values	Role	Consequence
`choice`	0 (orange) or 1 (blue)	Product in use	Counts for the market share. It is what friends observe when they compute the norm
`satisfaction`	log10(1 + 9q) of the product in use	Probability of keeping the product (habit)	With q = 0.5 it equals 0.74. Higher quality produces stronger habits and less attention to the alternative
`personal_experience`	Remembered quality of each product, initially [0, 0]	Memory of the consumer	A product never used is valued at 0, and a change in the quality of the unused product is not perceived
`utilities`	Total utility of each product	Basis of the choice when the consumer neither keeps nor explores	Combines norm and product utility (Sect. 2.4)
`friends`	List of linked consumers	Social network of the consumer	Source of information and of the norm

```
17  class Consumer(mesa.Agent):
18      """A consumer who uses one of two products and has friends."""
19
20      def __init__(self, model, choice, pos):
21          super().__init__(model)
22          # product in use: 0 orange, 1 blue
23          self.choice = choice
24          # total utility of each product
25          self.utilities = [1.0, 0.0] if choice == ORANGE else [0.0, 1.0]
26          # probability of keeping the product (habit)
27          self.satisfaction = model.satisfaction(choice)
28          # remembered quality of each product
29          self.personal_experience = [0.0, 0.0]
30          # location, used only to build the network
31          self.pos = pos
32          # linked consumers (social network)
33          self.friends = []
```

Listing 2. Consumer state variables (`consumer_choice/model.py`, lines 17 to 33).

2.3 Social Network

Each consumer is placed at a random position in a 51 × 51 space. Links are created one at a time: a random consumer is linked to the nearest consumer to which it is not yet linked, until the number of links equals `avg_connections` × `num_consumers` / 2 (250 in the baseline). The position is used only for this purpose (Listing 3). Because links connect nearby consumers, the network is local: friends of a consumer tend to be friends of each other. This property favours the formation of clusters of consumers that use the same product.

```
108  def build_network(self, avg_connections):
109      """Link a random consumer to its nearest unlinked consumer."""
110      num_links = (avg_connections * self.num_consumers) / 2
111      links = 0
112      while links < num_links:
113          a = self.random.choice(self.consumers)
114          cands = [c for c in self.consumers
115                   if c is not a and c not in a.friends]
116          if not cands:
117              continue
118          d_min = min(math.dist(a.pos, c.pos) for c in cands)
119          nearest = [c for c in cands
120                     if math.dist(a.pos, c.pos) == d_min]
121          target = self.random.choice(nearest)
122          a.friends.append(target)
123          target.friends.append(a)
124          links += 1
```

Listing 3. Construction of the social network (`consumer_choice/model.py`, lines 108 to 124).

2.4 Actions of the Consumer

Consumption Decision. In each tick the consumer applies three rules in sequence (Listing 4). First, with probability equal to `satisfaction`, it keeps the current product (habit). Second, if it did not keep the product, it switches to the other product with probability `exploration`. Third, if it neither kept nor explored, it chooses the product with the higher utility. If both utilities are equal, it keeps the current product. Satisfaction is then recomputed for the product in use. In the baseline the probability of exploring in a given tick is $(1 - 0.74) \times 0.01 \approx 0.003$.

```
 35  def consume(self):
 36      """Keep the product, explore the other, or choose by utility."""
 37      if self.random.random() < self.satisfaction:
 38          pass                                # habit: keep the product
 39      elif self.random.random() < self.model.exploration:
 40          self.choice = 1 - self.choice       # explore the other product
 41      elif self.utilities[ORANGE] != self.utilities[BLUE]:
 42          self.choice = (ORANGE if self.utilities[ORANGE]
 43                         > self.utilities[BLUE] else BLUE)
 44      self.satisfaction = self.model.satisfaction(self.choice)
     ...
104  def satisfaction(self, choice):
105      """Satisfaction grows with the log of quality: log10(1 + 9q)."""
106      return math.log10(1 + 9 * self.quality[choice])
```

Listing 4. Consumption decision and satisfaction (`consumer_choice/model.py`, lines 35 to 44, 104 to 106).

Utility Update. After the consumption phase, each consumer computes the utility of each product (Listing 5) in four steps: (1) the experience of the product in use is set to its current quality, (2) with probability `information_exchange`, and only if at least one friend uses the product, the consumer receives the current quality of that product, (3) the norm-based utility is the share of friends who use the product, and (4) the total utility is

$$U = w \cdot N + (1 - w) \cdot (E + I) / 2 ,$$

where w is `norm_influence`, N the share of friends using the product, E the remembered quality and I the quality reported by friends (0 if no information was received). Without information from friends, the product utility is half of the remembered quality. As a result, information exchange favours the product that friends already use, in the same direction as the norm.

```
46  def update_utilities(self):
47      """Combine own experience, friends' information and the norm."""
48      m = self.model
49      for option in (ORANGE, BLUE):
50          # 1. own experience: only the product in use is updated
51          if self.choice == option:
52              self.personal_experience[option] = m.quality[option]
53          # 2. information exchange: a friend who uses the product
54          #    reports its current quality
55          neighbor_info = 0.0
56          if (self.random.random() <= m.information_exchange
57                  and any(f.choice == option for f in self.friends)):
58              neighbor_info = m.quality[option]
59          # 3. social norm: share of friends who use the product
60          norm_util = 0.0
61          if self.friends:
62              users = sum(f.choice == option for f in self.friends)
63              norm_util = users / len(self.friends)
64          # 4. total utility: norm-based and product utility
65          prod_util = (self.personal_experience[option]
66                       + neighbor_info) / 2
67          self.utilities[option] = (
68              m.norm_influence * norm_util
69              + (1 - m.norm_influence) * prod_util)
```

Listing 5. Utility update (`consumer_choice/model.py`, lines 46 to 69).

Table 3 summarises the actions and their consequences.

Table 3. Actions of the consumer, in the order in which they are executed.

Action	When	Consequence
Keep the product (habit)	With probability `satisfaction`	No change. The consumer does not consider the alternative, so improvements of the other product are not perceived
Explore	If not kept, with probability `exploration`	Switches to the other product and learns its actual quality in the next update. It is the only way to discover an improvement of a product that no friend uses
Choose by utility	If neither kept nor explored, and utilities differ	Selects the product with the higher utility. In a tie it keeps the current product
Update experience	Every tick, for the product in use	Memory of that product equals its current quality
Ask friends	For each product, with probability `information_exchange`, if a friend uses it	Receives the current quality. Reinforces the product used by friends
Observe the norm	Every tick	Pulls utility towards the local majority. Produces clusters and can produce lock-in

Taken together, habit slows down change, exploration is the only source of new information about an unused product, and information exchange and the norm reinforce the product that is already used by friends. These mechanisms produce the three patterns described by the authors of the model: clustering of consumption, amplification of small differences in quality and social lock-in [1].

2.5 Global Parameters and Baseline

The model has eight parameters (Listing 6). The baseline uses the default values of the original model file (Table 4).

```
75  def __init__(self, num_consumers=100, avg_connections=5,
76               initial_pct_orange=50, norm_influence=0.15,
77               information_exchange=0.05, exploration=0.01,
78               quality_orange=0.5, quality_blue=0.5, rng=None):
79      super().__init__(rng=rng)
80      self.num_consumers = num_consumers
81      self.norm_influence = norm_influence
82      self.information_exchange = information_exchange
83      self.exploration = exploration
84      self.quality = [quality_orange, quality_blue]
```

Listing 6. Model parameters and their baseline values (`consumer_choice/model.py`, lines 75 to 84).

Table 4. Global parameters, baseline values and role. The last column summarises the effect measured in Sect. 3.3. Parameters marked "not tested" were kept at the baseline value.

Parameter	Baseline	Role	Effect of increasing the value
`num_consumers`	100	Size of the market	Not tested
`avg_connections`	5	Average number of friends	Not tested
`initial_pct_orange`	50	Initial share of orange consumers	Not tested
`norm_influence`	0.15	Weight of the norm against product utility	From 0 to 0.1, mean clustering rises from 0.54 to 0.94. Above 0.1 it changes little
`information_exchange`	0.05	Probability of learning from friends	Not tested
`exploration`	0.01	Probability of trying the other product	At 0.2, mean clustering falls to 0.67 and the dispersion of market shares decreases
`quality_orange`	0.5	Quality of the orange product	Not tested (reference value)
`quality_blue`	0.5	Quality of the blue product	A difference of 0.05 gives the better product 93% to 95% of the market (mean)

2.6 Model Outputs

The original model reports two plots: the market shares of the two products and the clustering of consumers. The implementation records the same three quantities at every tick with a Mesa `DataCollector` (Listing 7, Table 5). The analysis in Section 3 uses only these outputs.

```
 97      self.datacollector = mesa.DataCollector(model_reporters={
 98          "orange": lambda m: m.count_choice(ORANGE),
 99          "blue": lambda m: m.count_choice(BLUE),
100          "clustering": lambda m: m.consumer_cluster(),
101      })
102      self.datacollector.collect(self)
    ...
132  def count_choice(self, product):
133      return sum(c.choice == product for c in self.consumers)
134
135  def consumer_cluster(self):
136      """Mean share of friends who use the consumer's product."""
137      total = 0.0
138      for c in self.consumers:
139          if c.friends:
140              same = sum(f.choice == c.choice for f in c.friends)
141              total += same / len(c.friends)
142      return total / self.num_consumers
```

Listing 7. Model outputs (`consumer_choice/model.py`, lines 97 to 102, 132 to 142).

Table 5. Outputs of the model.

Output	Definition	Interpretation
Orange consumers	Number of consumers with `choice` = 0	Market share of the orange product (out of 100)
Blue consumers	Number of consumers with `choice` = 1	Market share of the blue product (out of 100)
Clustering	Mean, over consumers, of the share of friends that use the same product	1: all friends use the same product. ≈0.5: choices independent of friends

2.7 Notes on the Implementation

Three differences between the description in the Info tab of the original model and its code were found. The implementation follows the code:

- The Info tab states a default `exploration` of 0.05, but the model file uses 0.01.
- The Info tab states that friends may share outdated information. In the code, a friend always reports the current quality of the product.
- The Info tab states that less satisfied consumers seek more information. In the code, the probability of information exchange does not depend on satisfaction. Satisfaction only determines whether the utilities are used in the decision.

3 Baseline Simulation and Statistical Analysis

3.1 Experimental Setup

The baseline was replicated 1000 times with different random seeds (0 to 999), for 2000 ticks each, with the outputs recorded at every tick. The statistics are reported at tick 1000, and ticks 1500 and 2000 are used to check the horizon. For the sensitivity analysis, three parameters were varied one at a time, with 100 replications per value and 1000 ticks per run (Table 6). The same 100 seeds were used for every value of a parameter (common random numbers), so that differences between values are not due to different random draws. The experiments use the Mesa `batch_run` function (Listing 8).

Table 6. Experimental design.

Experiment	Parameter values	Runs per value	Ticks
Baseline	Table 4	1000	2000
Norm	`norm_influence` = 0, 0.1, …, 1.0	100	1000
Quality	`quality_blue` = 0.3, 0.4, 0.45, 0.5, 0.55, 0.6, 0.7	100	1000
Exploration	`exploration` = 0, 0.01, 0.05, 0.1, 0.2	100	1000

```
24  BASELINE = dict(num_consumers=100, avg_connections=5,
25                  initial_pct_orange=50, norm_influence=0.15,
26                  information_exchange=0.05, exploration=0.01,
27                  quality_orange=0.5, quality_blue=0.5)
    ...
49      results = batch_run(ConsumerChoiceModel, parameters=params,
50                          rng=list(range(seed0, seed0 + runs)),
51                          max_steps=ticks,
52                          data_collection_period=1 if every_tick else -1,
53                          number_processes=PROCESSES,
54                          display_progress=False)
```

Listing 8. Baseline values and batch run (`experiments/run_experiments.py`, lines 24 to 27, 49 to 54).

For each output we report the mean, the standard deviation (SD), the 95% confidence interval (CI) of the mean (Student's *t*), the median and the 5th and 95th percentiles (P5, P95). Market shares are expressed as fractions of the 100 consumers.

3.2 Baseline Results

Table 7 reports the statistics of the three outputs at tick 1000 over the 1000 runs, and Fig. 1 shows their evolution.

Table 7. Baseline outputs at tick 1000 (1000 runs).

Output	Mean	SD	95% CI	Median	P5	P95
Orange market share	0.506	0.252	0.490 to 0.521	0.510	0.11	0.92
Blue market share	0.494	0.252	0.479 to 0.510	0.490	0.08	0.89
Clustering	0.937	0.031	0.935 to 0.939	0.941	0.88	0.98

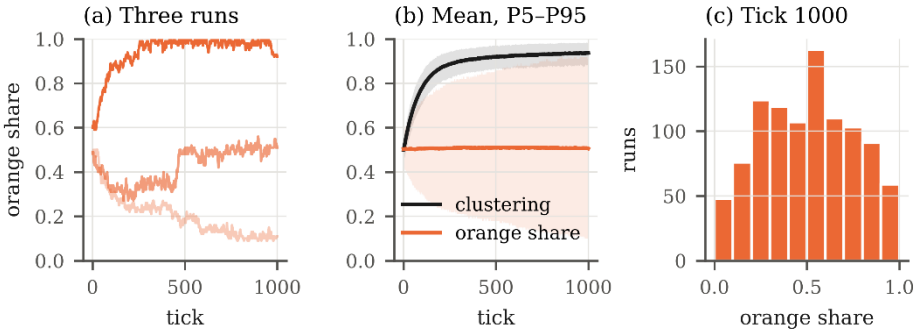


Fig. 1. Baseline. (a) Orange market share in three runs (the 5th, 50th and 95th percentile of the final share). (b) Mean clustering and mean orange share over 1000 runs, with the band between P5 and P95. (c) Distribution of the orange share at tick 1000.

Three results characterise the baseline:

- *Each market becomes segmented.* Mean clustering rises from 0.50 at tick 0 to 0.94 at tick 1000 (95% CI 0.935 to 0.939). On average, 94% of the friends of a consumer

use the same product. The variation between runs is small (SD 0.031), so this result does not depend on the random seed.

- *The market outcome is not predictable.* The mean orange share is 0.51, as expected from the symmetry of the baseline (equal qualities and initial shares), but the SD between runs is 0.25. In 90% of the runs the orange share lies between 0.11 and 0.92. Orange holds the majority in 50.5% of the runs and blue in 47.4% (ties: 2.1%). Only 27% of the runs end with shares between 0.40 and 0.60, and 10.8% end with one product at 90% or more.
- *The mean share is not informative, the dispersion is.* With identical products, the result of a run depends on early random events that are amplified by the norm and by information exchange. The relevant statistic for the baseline is therefore the distribution of shares (Fig. 1c), not its mean.

Table 8. Mean and SD of the outputs over time (baseline).

Tick	Clustering (mean)	Clustering (SD)	Orange share (mean)	Orange share (SD)
0	0.499	0.033	0.503	0.051
100	0.779	0.043	0.505	0.128
250	0.887	0.036	0.508	0.176
500	0.919	0.031	0.510	0.210
750	0.931	0.032	0.509	0.235
1000	0.937	0.031	0.506	0.252
1500	0.944	0.028	0.510	0.274
2000	0.948	0.031	0.511	0.291

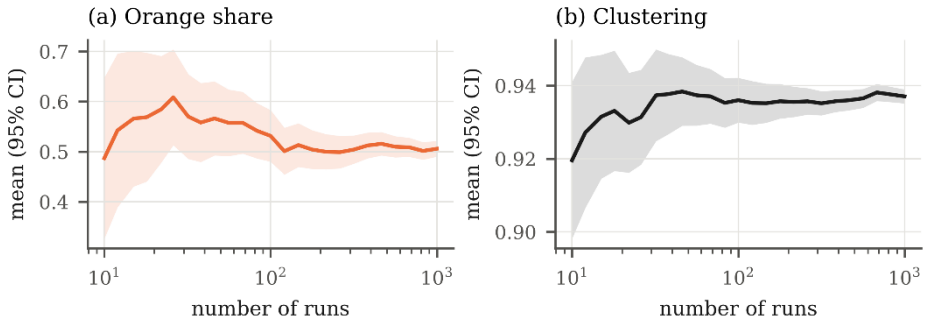


Fig. 2. Mean of the outputs at tick 1000 as a function of the number of runs, with the 95% confidence interval. (a) Orange market share. (b) Clustering.

Number of Runs. Figure 2 shows the mean of each output at tick 1000 as a function of the number of runs. With 100 runs, the half-width of the 95% confidence interval is 0.051 for the orange share and 0.006 for clustering. With 1000 runs it is 0.016 and 0.002. The market share requires more runs than clustering because its variance between runs is about 67 times larger. We therefore use 1000 runs for the baseline and 100 runs per value in the sensitivity analysis, where the differences between parameter values are larger than this precision.

Horizon. Table 8 shows the outputs at several ticks. Mean clustering increases by 0.018 between ticks 500 and 1000 and by 0.011 between ticks 1000 and 2000. The SD of the orange share keeps increasing (0.21, 0.25, 0.29 at ticks 500, 1000 and 2000). The model does not reach a stationary distribution of market shares within 2000 ticks: individual markets continue to move away from the initial 50/50 split. Tick 1000 is used as the reference horizon because the change of mean clustering after it is small and the shares have already separated. Statistics at a later tick would show a larger dispersion of shares, and this must be considered when the extended model is compared with the baseline.

3.3 Sensitivity Analysis

The sensitivity analysis varies one parameter at a time, with all other parameters at the baseline. Figures 3 to 5 show the distribution of the orange share and the mean clustering at tick 1000, and Tables 9 to 11 report the statistics.

Social Norm. Without norm (`norm_influence` = 0), mean clustering is 0.54, close to the value of independent choices, and the orange share stays near 0.5 (SD 0.07). With `norm_influence` = 0.1, mean clustering is 0.94 and the SD of the orange share is 0.24. Above 0.1 both values change little (clustering between 0.94 and 0.95). The effect of the norm is a threshold, not a gradual effect. The reason is visible in the utility function. With both qualities equal to q, the product utility of a consumer can only take the values 0, $q/2$ and q, so the difference between the two products is either 0 or at least $q/2$. The norm can reverse a non-zero difference only if $w / (1 - w) > q/2$, that is, $w > q / (2 + q) = 0.2$ for $q = 0.5$. For $w \leq 0.2$ the norm acts only when the product utilities are equal. This explains why the results for $w = 0.1$ and $w = 0.2$ are identical under common random numbers (Table 9), and why a small positive norm is sufficient to coordinate the choices of friends.

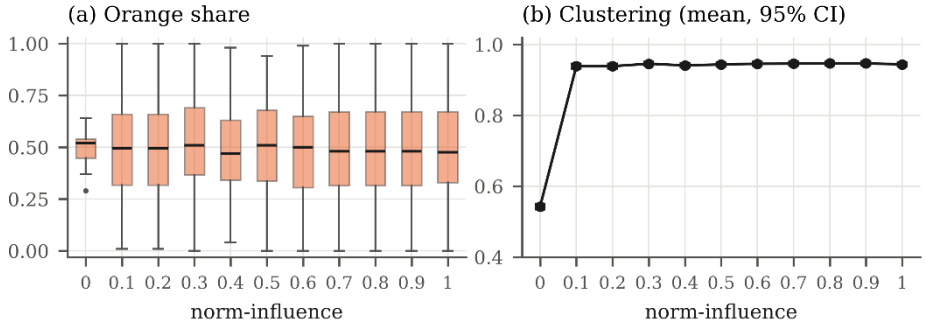


Fig. 3. Effect of `norm_influence` (100 runs per value). (a) Distribution of the orange share at tick 1000. (b) Mean clustering at tick 1000 with 95% confidence interval.

Table 9. Outputs at tick 1000 as a function of `norm_influence` (100 runs per value).

`norm_influence`	Orange share (mean)	Orange share (SD)	Clustering (mean)	Clustering 95% CI
0	0.499	0.067	0.543	0.535 to 0.550
0.1	0.506	0.238	0.939	0.932 to 0.945
0.2	0.506	0.238	0.939	0.932 to 0.945
0.3	0.524	0.239	0.946	0.940 to 0.951
0.4	0.490	0.209	0.941	0.936 to 0.946
0.5	0.507	0.232	0.944	0.939 to 0.948
0.6	0.492	0.231	0.946	0.941 to 0.951
0.7	0.499	0.234	0.946	0.941 to 0.951
0.8	0.504	0.237	0.947	0.941 to 0.952
0.9	0.503	0.239	0.947	0.942 to 0.952
1	0.504	0.235	0.944	0.939 to 0.949

Product Quality. With `quality_orange` fixed at 0.5, a small difference in quality determines the market. With `quality_blue` = 0.45, the mean orange share is 0.95. With 0.55, it is 0.07, which gives blue a mean share of 0.93. With a difference of 0.1 or more, the better product holds at least 98.8% of the market in the mean. Mean clustering is higher when qualities differ (0.98 at 0.3 and 0.99 at 0.7) than when they are equal (0.94), because most consumers then use the same product. The dispersion of the outcome is largest when the products are equal (SD 0.25) and falls to 0.014 or less when the difference is 0.1 or more.

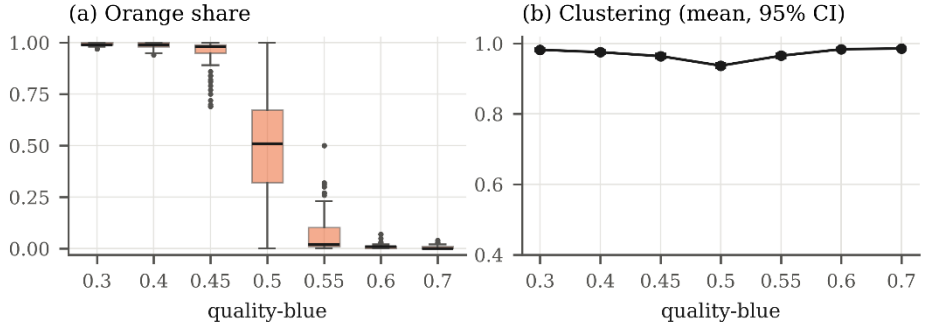


Fig. 4. Effect of `quality_blue`, with `quality_orange` = 0.5 (100 runs per value). (a) Distribution of the orange share at tick 1000. (b) Mean clustering at tick 1000 with 95% confidence interval.

Table 10. Outputs at tick 1000 as a function of `quality_blue` (`quality_orange` = 0.5, 100 runs per value).

`quality_blue`	Orange share (mean)	Orange share (SD)	Clustering (mean)	Clustering 95% CI
0.3	0.992	0.008	0.982	0.979 to 0.985
0.4	0.988	0.013	0.975	0.971 to 0.980
0.45	0.951	0.074	0.964	0.958 to 0.970
0.5	0.508	0.251	0.937	0.931 to 0.943
0.55	0.071	0.107	0.965	0.960 to 0.971
0.6	0.010	0.014	0.984	0.980 to 0.987
0.7	0.007	0.009	0.986	0.983 to 0.989

Exploration. Mean clustering decreases as exploration increases, from 0.94 at 0.01 to 0.86 at 0.05 and 0.67 at 0.2, because random switches break the agreement between friends. The dispersion of the orange share is not monotonic: its SD is 0.18 without exploration, 0.26 at 0.05 and 0.12 at 0.2. A possible interpretation is that, without exploration, few consumers change product and the shares move slowly. Moderate exploration increases the number of changes and lets clusters grow, and high exploration randomises choices and keeps the shares close to 0.5.

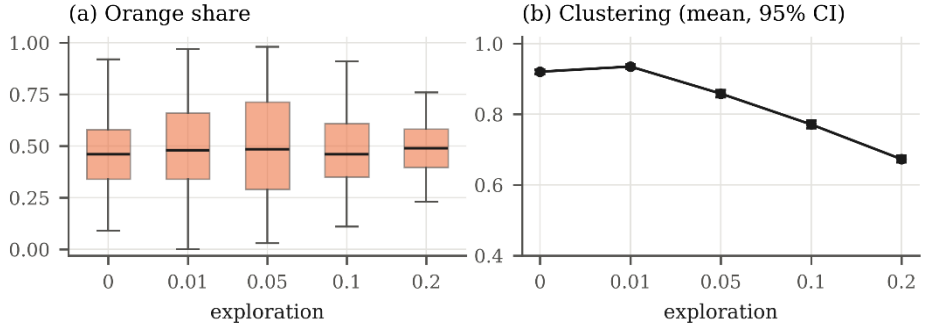


Fig. 5. Effect of `exploration` (100 runs per value). (a) Distribution of the orange share at tick 1000. (b) Mean clustering at tick 1000 with 95% confidence interval.

Table 11. Outputs at tick 1000 as a function of `exploration` (100 runs per value).

`exploration`	Orange share (mean)	Orange share (SD)	Clustering (mean)	Clustering 95% CI
0	0.468	0.177	0.920	0.915 to 0.926
0.01	0.493	0.229	0.935	0.929 to 0.941
0.05	0.499	0.259	0.858	0.848 to 0.868
0.1	0.483	0.181	0.771	0.761 to 0.781
0.2	0.488	0.121	0.673	0.664 to 0.682

3.4 Summary of Findings

The baseline and the sensitivity analysis led to four conclusions that will serve as reference for the extended model:

- Social influence segments the market. All tested positive values of `norm_influence` (0.1 to 1.0) lead to a mean clustering of about 0.94. Without norm, clustering stays at 0.54.
- With equal products, the outcome of a market is path dependent: the same rules produce markets dominated by either product (SD of the share 0.25). The baseline must be described by the distribution of outcomes, not by the mean.
- Small differences in quality are amplified: a difference of 0.05 gives the better product about 93% to 95% of the market.
- Exploration is the only mechanism that brings new information into a cluster. It reduces clustering but also determines how fast markets can change.

4 The New Model: DIGI's Entry into the Portuguese Market

4.1 Problem and Data

In the third quarter of 2024 the market of bundled services was shared by MEO (41.8%), NOS (35.0%), Vodafone (20.6%) and NOWO (2.6%) [6]. DIGI bought NOWO and started selling in November 2024 at €22 per month for fibre and television, with a commitment of 3 months [9]. The main offers of the incumbents cost €40.50 to €42.49 with contracts of 24 months [10]. In this report the list price of an operator is the monthly price of its main offer of fibre and television, as published before the entry of DIGI: €42.49 for MEO and NOS and €40.50 for Vodafone. A household pays the list price when it signs a new contract (renewals receive a discount, Sect. 4.2). The price cuts tested in Sections 4.4 and 5 are fractions of the list price, from ×0.95 to ×0.80. In the first quarter of 2026, 16 months after the entry, DIGI and NOWO together held 3.2% [6]. The incumbents kept the prices of their main brands almost unchanged [10]. A cheaper operator was therefore not enough to move the market quickly. Agent-based models have been proposed for telecommunications markets because demand there is non-linear and shaped by interactions between customers, such as word of mouth, that aggregate demand equations do not capture [11]. NOWO already charged much less than the incumbents before the entry (€26.35 [12]) and held only 2.6%.

The new model is used to answer two questions. Which mechanisms explain the slow growth of a much cheaper operator? And what is the best price response of the incumbents when each of them learns from its own profit? Table 12 summarises the data used for each operator.

Table 12. Operators in the model. Shares: ANACOM, third quarter of 2024 [6]. Prices of fibre and television with VAT (value added tax) [9,10,12]. Complaints per thousand customers in 2025 [6]. Quality is derived from the complaints of 2025 and of the second quarter of 2026 (Sect. 4.2).

Operator	Initial share	Price	Contract	Complaints	Quality
MEO	41.8%	€42.49	24 months	4.0	0.848
NOS	35.0%	€42.49	24 months	6.2	0.784
Vodafone	20.6%	€40.50	24 months	7.7	0.776
NOWO, before the entry	2.6%	€26.35	24 months	18.7	0.572
DIGI, from month 12	NOWO's customers	€22.00	3 months	18.7	0.572 to 0.756

4.2 From the Baseline to the New Model

The decision rule of the baseline is kept: each month a consumer keeps its operator by habit with probability equal to its satisfaction, otherwise explores another operator with probability `exploration`, otherwise chooses the operator with the highest utility. The network, the memory of quality and the information from friends are also unchanged. With two operators and all extensions switched off, the new model reproduces the

baseline exactly, with the same random draws. Table 13 lists what changes. One tick is one month.

Table 13. Correspondence between the original model and the new model.

Original model	New model
2 products	4 operators. NOWO becomes DIGI at month 12 (November 2024) and keeps its customers
Fixed quality	Quality from complaints. DIGI improves during its first 18 months
No price	Price in the product utility, with a weight taken from a choice experiment [13]
Habit	Habit and loyalty contracts with the legal penalty for early exit [14]
No renewal	At the end of a contract the consumer decides again. If it stays, it may accept a renewal discount
100 consumers, fixed	At least 1000 households. 0.5% leave each month and new households join
Fixed products	The three incumbents learn their prices after the entry (Sect. 4.4)

Utility with Price. The utility of operator k for a consumer is

$$U_k = w \cdot N_k + (1 - w) \cdot [\, (E_k + I_k) / 2 - \theta \cdot p_k \,] + \beta_k \,,$$

where the first terms are those of the baseline, p_k is the price the consumer pays (the contract price for its own operator, the current price for the others) and β_k is a brand constant (Listing 9). The price weight θ is not free. Confraria et al. [13] estimate with a discrete choice experiment that Portuguese consumers value one unit of market share as much as €10.22 per month. Imposing the same ratio between the norm and the price gives $\theta = w / ((1 - w) \cdot 10.22)$, which is 0.0109 per euro for $w = 0.10$.

```
126  price = self.contract_price if option == self.choice else m.price[option]
127  prod_util = ((self.personal_experience[option] + neighbor_info) / 2
128               - m.price_weight * price)
129  self.utilities[option] = (m.norm_influence * norm_util
130                            + (1 - m.norm_influence) * prod_util
131                            + m.brand_constant[option])
```

Listing 9. Utility with price and brand constant (`consumer_choice/telecom.py`, lines 126 to 131).

Quality from Complaints. The model needs a quality between 0 and 1, and ANACOM publishes complaints per thousand customers [6]. With r the ratio of an operator's complaints to the mean of the incumbents, quality is $q = 0.80 - \kappa \ln r$, with $\kappa = 0.20$. For the incumbents, r is the mean of the ratios in 2025 and in the second quarter of 2026. Doubling the complaints therefore always lowers quality by the same amount. The value of κ is a modelling choice and is tested in Sect. 5.4. The complaints of DIGI fell from 3.1 to 1.25 times the incumbents' mean in about 18 months while its share barely changed, so its quality rises linearly with the time since entry, from 0.572 to 0.756.

Contracts. A consumer who chooses an operator signs a contract at the current price and pays that price until the contract ends. Before the end it can leave only if the savings cover the legal penalty, which is 50% of the remaining fees in the first year and 30% in the second [14] (Listing 10). A scenario with full lock-in is also tested. When the contract ends, the consumer decides as in any other month and can leave without penalty. If it stays, the operator offers a renewal discount of 10% for a new contract of 24 months. The consumer accepts if the discount is at least its own cost of being committed. This cost comes from the same choice experiment [13], which values a shorter commitment at about €0.22 per month of commitment, so about €5.25 per month for 24 months. Each consumer draws its cost uniformly between 50% and 150% of this value. DIGI offers no renewal discount.

```
72  def can_leave(self, target):
73      """Early exit allowed only if the savings cover the legal penalty."""
74      m = self.model
75      if not m.contracts or self.contract_left <= 0:
76          return True
77      if m.exit_rule == "lock":
78          return False
79      months_left = self.contract_left
80      rate = 0.50 if self.contract_elapsed < 12 else 0.30      # Lei 16/2022
81      penalty = rate * months_left * self.contract_price
82      savings = months_left * (self.contract_price - m.price[target])
83      return savings - penalty > 0
```

Listing 10. Early exit from a contract (`consumer_choice/telecom.py`, lines 72 to 83).

Population. The number of bundle subscribers grows about 1% per year [6]. Each month 0.5% of the households leave and slightly more join, with a minimum of 1000. A new household links to its nearest neighbours, chooses by utility with what its friends tell it and signs a contract with the chosen operator. New households are an important channel for DIGI, because they choose before any contract binds them. The exit rate is not observed and is tested in Sect. 5.4.

4.3 Calibration and Validation

Seven values were fixed from data or from the literature: the initial shares, the prices, the contract lengths, the qualities, the price weight, the commitment cost and the legal penalties. The main assumptions without data are that 70% of the households start under

contract, that the renewal discount is 10% and that 0.5% of the households leave each month. The remaining values were calibrated in two steps.

- *Before the entry.* Without brand constants the norm favours the largest operator and the shares drift. Constants were calibrated so that, without DIGI, the shares stay at the ANACOM values (MEO is the reference, NOS +0.0307, Vodafone +0.0161, NOWO and DIGI −0.0820). The share of initial contracts that already carry the renewal discount (48%) was set to its own long run value, so that the average price paid does not drift.
- *After the entry.* A grid over the norm weight $w \in \{0.08, 0.10, 0.12\}$ and the exploration rate $\in \{0.005, 0.01, 0.02, 0.03\}$, with 24 runs per point, minimised the error against the DIGI shares at months 19, 25 and 28. The point $w = 0.10$ and exploration 0.02 was kept. It is the second best point (RMSE 0.15 against 0.12) but keeps the norm weight $w = 0.10$ of the earlier calibration and gives more switching between operators (2.7% of the households per year against 2.1%), which the model otherwise underestimates (Sect. 6.3).

With the final values and 30 runs, the model gives DIGI 2.4%, 2.9% and 3.2% at months 19, 25 and 28, against 2.8%, 3.0% and 3.2% in the data (RMSE 0.22 percentage points, Fig. 6). At month 28 the incumbents hold 40.3%, 35.1% and 21.4%, against 41.5%, 34.9% and 20.3%. The model therefore reproduces the main fact: half the price is not enough for fast growth, because most households are under contract, keep their operator by habit and follow their friends. This agrees with studies of telecommunications markets in which few consumers change operator and many do not reach the cheapest offer even when they change plan [15]. Two limitations remain. The model has fewer switches at the end of a contract than the survey of Confraria et al. suggests, so DIGI grows through a narrower channel than in reality. Without the entry, MEO also loses share slowly (41.5% at month 0, against 41.8% in the data, and 38.1% at month 132). This drift is common to all scenarios, which are always compared with each other under the same random numbers.

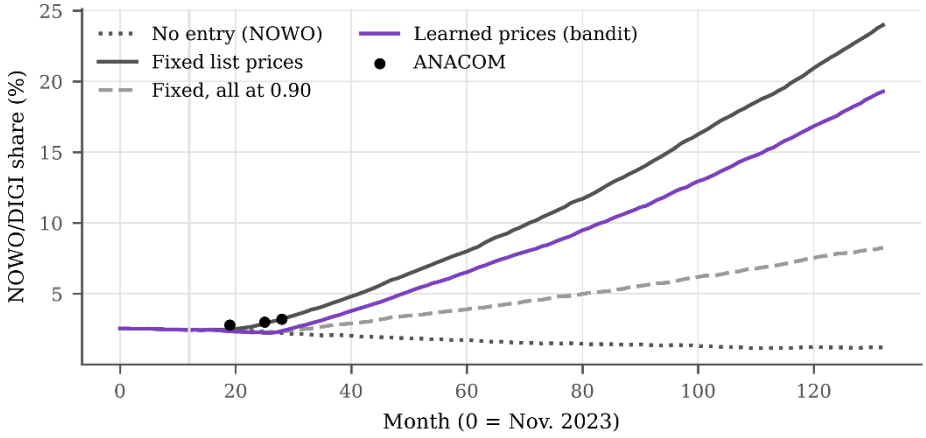


Fig. 6. Share of NOWO and DIGI (mean of 30 runs) with the ANACOM data, and its evolution over 10 years under fixed list prices, learned prices, a common cut to 0.90 and no entry. Data exist only up to month 28.

4.4 Learning Incumbents

From the month of the entry, MEO, NOS and Vodafone learn their prices as independent agents (task 5). Each one uses an ε-greedy multi-armed bandit [16]. Every 3 months it chooses a price multiplier from {1.00, 0.95, 0.90, 0.85, 0.80} of its list price. The lowest price, €34 for MEO and NOS, stays above the unit cost. The reward is the profit of the quarter, the sum over its customers of the price paid minus the unit cost. The unit cost is 60% of the list price, a margin of 40% close to the 2024 EBITDA (earnings before interest, taxes, depreciation and amortisation) margins of NOS (45.3%) and MEO (35.8%). With probability $\varepsilon = 0.1$ the incumbent tries a random price, otherwise it chooses the price with the highest value. After each quarter it updates the value of the price it used:

$$Q(a) \leftarrow Q(a) + \alpha \cdot [\, R - Q(a) \,] , \quad \alpha = 0.2 .$$

A constant step α gives more weight to recent quarters, which suits a market that keeps changing as DIGI grows and the rivals learn. In the first quarter after the entry the incumbent keeps the list price. It then tries each other price once, in random order, and the first reward of each price becomes its value (Listing 11). An incumbent sees only its own profit. It does not observe the prices of its rivals and cannot communicate with them. DIGI keeps its price of €22. The bandit draws from its own random generator, so the consumers receive exactly the same random numbers as in the scenarios with fixed prices.

```
75  reward = self.acc[k]
76  q = self.q[k]
77  a = self.action[k]
78  if q[a] is None:
79      q[a] = reward                              # first reward of this arm
80  else:
81      q[a] += self.alpha * (reward - q[a])
82  untried = [i for i, v in enumerate(q) if v is None]
83  if untried:
84      new_a = self.random.choice(untried)
85      explored = True
86  elif self.random.random() < self.epsilon:
87      new_a = self.random.randrange(len(self.arms))
88      explored = True
89  else:
90      best = max(q)
91      new_a = self.random.choice([i for i, v in enumerate(q) if v == best])
92      explored = False
```

Listing 11. Update and choice of the bandit (`consumer_choice/learning.py`, lines 75 to 92).

4.5 Experimental Design

Each scenario was run 30 times for 132 months, 12 before the entry and 10 years after it, with the same seeds in every scenario (common random numbers). Profit is summed over the 10 years after the entry and compared with the scenario where all incumbents keep the list price. Differences are computed seed by seed (paired comparison) and reported with 95% confidence intervals. The scenarios are listed in Table 14. The 10 year values compare scenarios under the same assumptions. They are not forecasts, because the data cover only the first 28 months.

Table 14. Scenarios of the new model.

Scenario	Prices of the incumbents after the entry	Question
No entry	List prices, NOWO stays	What does the entry cost?
All at list price	×1.00 (reference)	Reference
Common cuts	All three at ×0.95, 0.90, 0.85 or 0.80	Does matching DIGI pay?
Solo cuts	One incumbent at ×0.90 or 0.80, the others at list price	Is the list price a best response?
Learned prices	Bandit, decisions every 3, 6 or 12 months	What do learning agents do?
Bandit parameters	$\varepsilon \in \{0.05, 0.10, 0.20\} \times \alpha \in \{0.1, 0.2, 0.4\}$	Do ε and α matter?

5 Results of the New Model

5.1 The Entry and Common Price Cuts

The entry costs the incumbents about a tenth of their profit. Without DIGI they would earn 11.4% more (95% CI 10.2 to 12.7) than with DIGI and list prices (Table 15). With list prices DIGI reaches 22.6% in year 10. Matching DIGI on price slows it down: if all incumbents cut to ×0.90, DIGI stays at 7.9%. But every common cut costs more than it saves, from −9.1% at ×0.95 to −41.2% at ×0.80. The customers kept by the cut do not compensate for the lower margin on all the others.

Table 15. Joint profit of the incumbents over 10 years after the entry, in % against all at list price (paired by seed, 30 runs), mean share of DIGI in year 10 and average monthly contract price in year 10, over all households (DIGI's customers included).

Scenario	Profit (%)	95% CI	DIGI (%)	Price (€)
No entry	+11.4	+10.2 to +12.7	n/a	40.86
All at list price	0.0	reference	22.6	36.71
Learned, every 3 months	−2.4	−3.5 to −1.3	18.1	37.10
Learned, every 6 months	−3.9	−4.7 to −3.1	16.3	37.52
Learned, every 12 months	−7.5	−8.4 to −6.7	12.4	38.53
All at ×0.95	−9.1	−9.8 to −8.4	16.7	36.24
All at ×0.90	−17.5	−18.2 to −16.8	7.9	36.07
All at ×0.85	−28.4	−29.2 to −27.6	1.9	35.07
All at ×0.80	−41.2	−41.8 to −40.7	0.8	33.33

5.2 Cutting Alone: a Prisoner's Dilemma

Table 16 shows the profit of each incumbent when it alone cuts its price to ×0.90 and when all three do. Each incumbent gains by cutting alone, by +14.9% (MEO), +19.3% (NOS) and +36.2% (Vodafone), and all three confidence intervals exclude zero. The gain comes from the rivals' customers, so joint profit still falls by 6.6% to 8.9%. When all three cut, each one loses. This is a prisoner's dilemma: the list price is not an

equilibrium, but among the symmetric prices tested it gives the highest joint profit. The result depends on the size of the cut. A deeper solo cut, to ×0.80, loses money for MEO (−11.1%) and NOS (−8.3%) and still pays for Vodafone (+13.1%), the incumbent with the smallest base of customers.

Table 16. Own profit of each incumbent over 10 years, in % against all at list price (95% CI).

Incumbent	Cuts alone to ×0.90	All cut to ×0.90
MEO	+14.9% (+12.6 to +17.1)	−18.3% (−19.6 to −16.9)
NOS	+19.3% (+17.1 to +21.6)	−8.7% (−10.0 to −7.4)
Vodafone	+36.2% (+31.8 to +40.7)	−30.5% (−32.5 to −28.6)

5.3 Learned Prices

The learning incumbents do not start a price war (Fig. 7). During the first 15 months after the entry each one tries every price once, and the mean price falls by 10% to 15%. After that they return close to the list price, with a mean multiplier of 0.98. In years 6 to 10 the list price is chosen in 85% (MEO), 88% (NOS) and 88% (Vodafone) of the decisions. Most of the remaining decisions are the random exploration: when the incumbents do not explore, they choose the list price 95% of the time. Joint profit is only 2.4% lower than with list prices (95% CI 1.3 to 3.5), much better than any common cut, and DIGI reaches 18.1% in year 10.

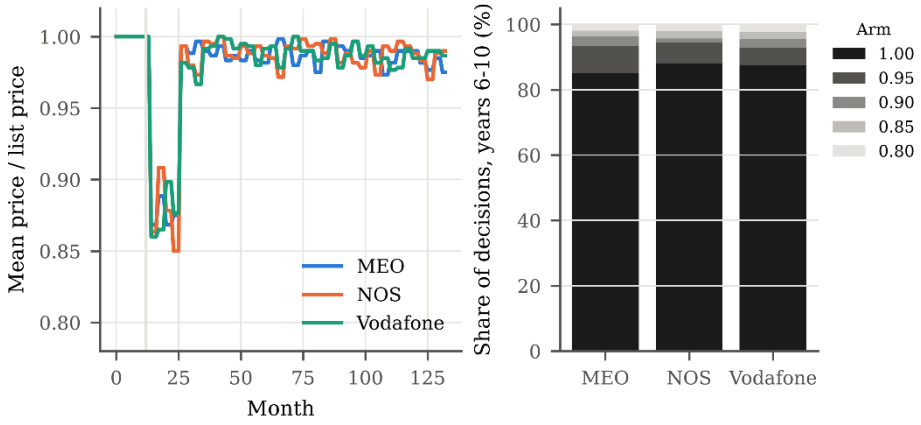


Fig. 7. Learned prices (ε = 0.1, α = 0.2, 30 runs). Left: mean price divided by the list price. Right: share of each price in the decisions of years 6 to 10.

This outcome is not learned collusion. Calvano et al. [17] show that Q-learning agents that observe each other's prices can learn to keep prices high and to punish deviations. Our bandits cannot see their rivals, so they cannot reward or punish them. Our explanation is the short reward. A price cut lowers the margin at once, while the customers it wins arrive slowly, because most households are under contracts of 24 months. A customer won by a cut also keeps paying in later quarters, and that profit is credited to whichever price is active then, often the list price. Judged on three months

of profit, a cut therefore looks worse than it is. The real incumbents also kept the prices of their main brands almost unchanged after the entry, although they compete with cheaper secondary brands, which the model does not include.

Two checks support this reading. First, the result does not depend on ε and α. Across the nine combinations of $\varepsilon \in \{0.05, 0.10, 0.20\}$ and $\alpha \in \{0.1, 0.2, 0.4\}$ joint profit stays between −2.2% and −4.4% and the mean multiplier between 0.97 and 0.98. The learning rate barely matters, and more exploration costs more only because it forces more random cuts. Second, with a longer decision period the bandits cut more often, as expected if a short reward hides part of the value of a cut. With decisions every 6 and 12 months the mean multiplier falls to 0.97 and 0.94, but joint profit also falls, by 3.9% and 7.5%. Each price is then held longer, and the bandit has fewer decisions to learn from. We did not isolate which of these effects dominates, so this check is consistent with our explanation but does not prove it.

5.4 Sensitivity Analysis

Six assumptions without direct data were changed one at a time, in nine variants and without recalibration (task 6). For each variant the list price scenario, the three solo cuts to ×0.90 and the bandit were run 20 times with common random numbers (Fig. 8, Table 17). Because of the smaller number of runs, the base row differs slightly from the results above.

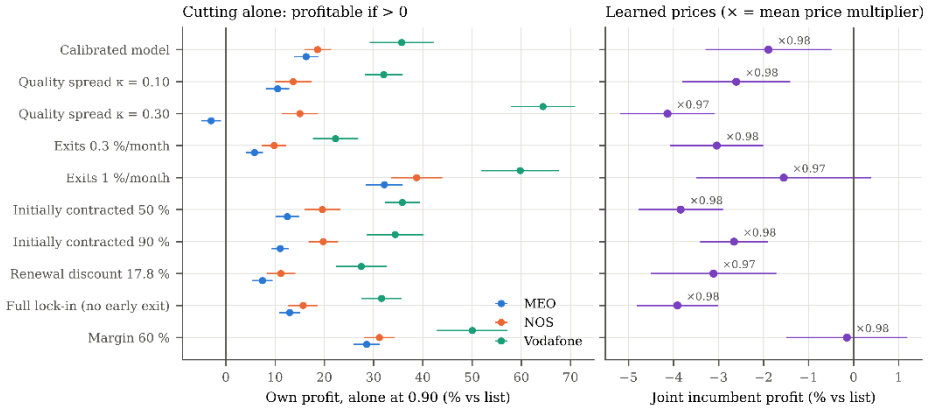


Fig. 8. Sensitivity of the two conclusions (20 runs per scenario, 95% CI). Left: own profit of an incumbent that alone cuts to ×0.90. Right: joint profit with learned prices and mean price multiplier.

Table 17. Sensitivity analysis (20 runs per scenario). Own profit of a solo cut to ×0.90 and joint profit with learned prices, in % against all at list price. DIGI share at month 28 (ANACOM: 3.2%) and in year 10 with list prices.

Variant	MEO solo	NOS solo	Vod. solo	Learned	DIGI m28	DIGI y10
Calibrated model	+16.3	+18.6	+35.7	−1.9	3.2	22.8
Quality spread κ = 0.10	+10.5	+13.7	+32.0	−2.6	4.8	27.0
Quality spread κ = 0.30	−3.0	+15.0	+64.4	−4.1	2.2	15.6
Exits 0.3%/month	+5.8	+9.8	+22.3	−3.0	3.2	18.4
Exits 1%/month	+32.2	+38.7	+59.8	−1.6	3.3	32.6
Initially contracted 50%	+12.5	+19.6	+35.8	−3.8	3.5	23.7
Initially contracted 90%	+11.0	+19.8	+34.4	−2.7	3.1	21.7
Renewal discount 17.8%	+7.4	+11.2	+27.5	−3.1	2.8	16.2
Full lock-in (no early exit)	+13.0	+15.7	+31.6	−3.9	2.8	16.2
Margin 60%	+28.6	+31.2	+50.0	−0.1	3.2	22.8

Both conclusions hold in all nine changes:

- *A solo cut pays for at least one incumbent.* NOS gains between +9.8% and +38.7% and Vodafone between +22.3% and +64.4%. The only cut that does not pay is MEO's with $\kappa = 0.30$ (−3.0%). A larger κ widens the quality gaps, and MEO, which has the best quality, already holds its customers.
- *Learned prices stay near the list price.* The mean multiplier stays between 0.97 and 0.98 and joint profit between −0.1% and −4.1%. With exits of 1% per month and with a margin of 60% the confidence interval includes zero.
- *Cuts pay more when more customers are free.* More exits bring more new households without a contract, and a higher margin makes each extra customer worth more. Both make solo cuts more profitable.

The long run share of DIGI is not robust. With list prices it ranges from 15.6% to 32.6% in year 10 across the variants. Without recalibration some variants also move away from the data at month 28 (4.85% with $\kappa = 0.10$ and 2.24% with $\kappa = 0.30$, against 3.2%). The analysis therefore tests the robustness of the conclusions, not the fit. The direction of the results is robust, but the exact share of DIGI in 10 years should not be read as a forecast.

5.5 Comparison with the Baseline

The new model keeps the main property of the baseline. Outcomes vary between runs with the same parameters: with list prices DIGI ends year 10 between 15.9% and 31.8% (standard deviation 3.8 points over 30 runs). As in the baseline, the distribution of outcomes matters, which is why every comparison is paired by seed. The norm also still groups friends. At month 0 clustering is 0.34, about the value expected if friends chose independently with the observed shares (0.34). By month 132 it rises to 0.54, while independent choices with the shares of that month would give only 0.26. Clustering stays below the 0.94 of the baseline because the market has four operators, contracts delay switching, households keep joining and leaving, and the horizon is 132 months instead of 1000 ticks. Habit and the norm, which in the baseline produced lock-in, now explain why a much cheaper operator grows slowly.

6 Conclusions

6.1 The New Model

- *Contracts, habit and friends slow a cheap rival.* The model reproduces the slow growth of DIGI (RMSE 0.22 points in the first 16 months after the entry) with the decision rule of the baseline, prices and legal contracts. Price alone does not move a market where most households are committed and follow their friends.
- *The list price is not an equilibrium.* Each incumbent gains by cutting its price alone, but all lose if all cut. Matching DIGI on price slows it down but costs much more than it saves.
- *Simple learners stay near the list price.* Bandits that see only their own quarterly profit return to the list price in most decisions and lose only 2.4% of joint profit. This happens without communication and without observing the rivals, and agrees with the observed behaviour of the incumbents. The reason in the model is the short reward horizon against contracts of 24 months, not collusion.
- *Both conclusions hold* in all nine variants of the assumptions that have no data (Sect. 5.4). The share of DIGI after 10 years does not: it ranges from 15.6% to 32.6% across the variants. It depends on these assumptions and should not be read as a forecast.

6.2 The Project as a Whole

Starting from a model of two abstract products, the project produced a model of a real market that is calibrated to public data and supported by a choice experiment, the law on contracts and the literature on learning and pricing. The decision rule of the original model was kept and remains testable, because the new model reproduces the baseline when the extensions are switched off. Three lessons stand out. First, statistics over many runs were needed at every stage, because single runs of both models can be very different. Second, paired comparisons with common random numbers made small differences between scenarios measurable. The first version of the bandit used the random generator of the consumers and broke this pairing until it was given its own generator. Third, the learning design matters: the first version tried the prices always in the same order and made the lower prices look worse than they were. Both errors were corrected, and the results above use the corrected version.

6.3 Limitations and Future Work

- The model has fewer switches at the end of a contract than the survey data suggest, and without the entry the shares drift slowly.
- Several values have no direct data (the initial share under contract, the renewal discount, the exit rate and the mapping from complaints to quality). They were tested in the sensitivity analysis, one at a time, so interactions between them are not known.

- The incumbents can only change the price of one offer. In reality they answered DIGI with cheaper secondary brands, and DIGI keeps a fixed price.
- The bandit has no state and a reward of one quarter. A learner that values future profit, such as Q-learning with the rivals' prices as state [17], could test whether the result changes when the incumbents can see each other.

References

1. Jager, W., Wang, S.: NetLogo Consumer Choice model. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL (2024). https://github.com/NetLogo/models/blob/master/Sample%20Models/Social%20Science/Consumer%20Choice.nlogox, last accessed 2026/11/07
2. Wilensky, U.: NetLogo. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL (1999). http://ccl.northwestern.edu/netlogo/, last accessed 2026/11/07
3. Janssen, M.A., Jager, W.: Fashions, Habits and Changing Preferences: Simulation of Psychological Factors Affecting Market Dynamics. Journal of Economic Psychology 22(6), 745–772 (2001)
4. Cialdini, R.B., Goldstein, N.J.: Social Influence: Compliance and Conformity. Annual Review of Psychology 55, 591–621 (2004)
5. Nyborg, K., et al.: Social Norms as Solutions. Science 354(6308), 42–43 (2016)
6. ANACOM: Statistical information on bundled services (third quarter of 2024 to first quarter of 2026) and on complaints (2025 and second quarter of 2026). Autoridade Nacional de Comunicações, Lisbon (2026). https://www.anacom.pt, last accessed 2026/11/07
7. ter Hoeven, E., Kwakkel, J., Hess, V., Pike, T., Wang, B., Rht, Kazil, J.: Mesa 3: Agent-Based Modeling with Python in 2025. Journal of Open Source Software 10(107), 7668 (2025). https://doi.org/10.21105/joss.07668
8. Grimm, V., et al.: The ODD Protocol for Describing Agent-Based and Other Simulation Models: A Second Update to Improve Clarity, Replication, and Structural Realism. Journal of Artificial Societies and Social Simulation 23(2), 7 (2020)
9. Observador: DIGI prices and conditions at launch in Portugal (4 November 2024). https://observador.pt/2024/11/04/digi-entra-em-portugal-sem-fidelizacao-no-servico-de-telemovel-so-internet-fixa-tem-periodo-de-retencao-de-tres-meses/, last accessed 2026/11/07
10. CNN Portugal, DECO PROteste: Barometer of telecommunications prices (11 September 2025). https://cnnportugal.iol.pt/digi/barometro/barometro-cnn-portugal-deco-proteste-esta-de-volta-e-ha-varias-descidas-e-subidas-de-preco-nas-telecomunicacoes/20250911/68c199f5d34e58bc67958dac, last accessed 2026/11/07
11. Twomey, P., Cadman, R.: Agent-Based Modelling of Customer Behaviour in the Telecoms and Media Markets. info 4(1), 56–63 (2002). https://doi.org/10.1108/14636690210426640
12. ANACOM: O Sector das Comunicações 2024, Table 20. Autoridade Nacional de Comunicações, Lisbon (2025). https://www.anacom.pt/render.jsp?contentId=1809701, last accessed 2026/11/07
13. Confraria, J., Ribeiro, T., Vasconcelos, H.: Analysis of Consumer Preferences for Mobile Telecom Plans Using a Discrete Choice Experiment. Telecommunications Policy 41(3), 157–169 (2017). https://doi.org/10.1016/j.telpol.2016.12.009
14. Lei n.º 16/2022, de 16 de agosto: Lei das Comunicações Eletrónicas. Diário da República. https://diariodarepublica.pt/dr/detalhe/lei/16-2022-187481298, last accessed 2026/11/07
15. Griffin, M., Lyons, S., Mohan, G., Joseph, M., Mac Domhnaill, C., Evans, J.: Intra-Operator Mobile Plan Switching: Evidence from Linked Survey and Billing Microdata. Telecommunications Policy 46, 102343 (2022). https://doi.org/10.1016/j.telpol.2022.102343

16. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge, MA (2018)
17. Calvano, E., Calzolari, G., Denicolò, V., Pastorello, S.: Artificial Intelligence, Algorithmic Pricing, and Collusion. American Economic Review 110(10), 3267–3297 (2020). https://doi.org/10.1257/aer.20190623